TacticalGrid

Modelado, búsqueda y decisión adversarial

Documento técnico del proyecto

Sistemas Inteligentes I

Universidad de Caldas · Manizales, Colombia

Integrantes

Paula Andrea Cano

Vanessa Alejandra Osorio

Versión 1.0 · 20 de octubre de 2026

Repositorio: TacticalGrid · Rama Union

Contenido

1. Resumen ejecutivo
2. Introducción
3. Arquitectura y flujo de una partida
4. Modelado formal del problema
5. Representación computacional del estado
6. Reglas del juego
7. Costos y función de transición
8. Algoritmos de búsqueda
9. Heurísticas
10. Decisión adversarial
11. Metodología experimental
12. Resultados
13. Análisis de complejidad
14. Limitaciones y trabajo futuro
15. Conclusiones
16. Referencias

Anexo A. Métricas crudas
Anexo B. Diagramas de los escenarios
Anexo C. Contrato del escenario JSON
Anexo D. Pseudocódigo
Anexo E. Trazabilidad, validación y uso de IA

Los marcadores del PDF permiten navegar directamente por las secciones. Las tablas y figuras se numeran en el cuerpo del documento; los anexos conservan la evidencia completa de la ejecución.

1. Resumen ejecutivo

TacticalGrid es un juego táctico bidimensional por turnos para dos bandos. Cada bando dispone de unidades sobre un tablero heterogéneo, debe recoger un recurso y llevarlo a su propia base; el rival puede interceptarlo, transferirlo entre aliados o bloquear su camino. El proyecto implementa el dominio del juego, la carga y validación de escenarios JSON, cinco estrategias de búsqueda no informada e informada (BFS, DFS, UCS, A* y Beam Search), dos estrategias de decisión adversarial (Minimax y poda alfa-beta), métricas de rendimiento y verificación de caminos.

La arquitectura está separada en paquetes: `domain` contiene el modelo puro; `io` valida los escenarios; `search` y `decision` implementan los algoritmos; `metrics` registra generados, expandidos, frontera, cortes y tiempos; `verification` comprueba los caminos; `app` orquesta la partida automática; y `ui` presenta el tablero con pygame. Esta separación permite razonar sobre el problema sin depender de la interfaz y facilita la experimentación.

Los experimentos se realizaron sobre cuatro escenarios: `example` (4×4, 2 unidades por bando), `jsonPrueba` (8×8, 3 por bando), `murosBandera` (4×4, 2 por bando) y `robo` (5×5, 3 por bando). Los resultados fueron consistentes: `example` terminó con victoria de B en 6 turnos; `jsonPrueba` con victoria de A en 18 turnos, salvo DFS, que necesitó 24; `murosBandera` terminó con B en 10; y `robo` con A en 5. Minimax y alfa-beta reprodujeron los mismos ganadores y turnos en los escenarios donde se ejecutaron.

El análisis muestra tres ideas centrales. Primera: el número de pasos y el costo de terreno no son lo mismo; DFS puede encontrar una ruta en más pasos y con mucho mayor costo, mientras UCS y A* minimizan el costo. Segunda: Beam Search reduce la exploración, pero puede perder la ruta óptima porque descarta alternativas sin lista de respaldo. Tercera: alfa-beta conserva la decisión de Minimax y reduce de forma notable los nodos evaluados y el tiempo. El documento también explicita las limitaciones actuales, en particular que la interfaz solo ofrece algoritmos de búsqueda y que no se ejecuta el bloque `prueba` del JSON.

> **Alcance de este documento**
> Se describe el comportamiento del código en la rama `Union`, con los cambios de ocupación de bases y transferencias ya integrados. No se presentan funciones hipotéticas: cuando una capacidad existe en el código pero no está expuesta en la interfaz, se indica expresamente.

2. Introducción

Contexto y motivación

El proyecto articula dos familias clásicas de la inteligencia artificial: la búsqueda en espacios de estados y la decisión en entornos adversariales. En un tablero pequeño, una unidad puede moverse, recoger un recurso, transferirlo a un aliado o interceptar a un enemigo. El estado cambia de forma discreta y determinista, pero las decisiones de ambos bandos se afectan mutuamente. Esa combinación permite comparar algoritmos con propiedades distintas: BFS y DFS exploran sin costos; UCS y A* incorporan pesos; Beam limita la frontera; Minimax y alfa-beta razonan sobre respuestas del rival.

La motivación didáctica es doble. Por un lado, el juego obliga a definir con precisión qué es un estado, qué es una acción legal y cuándo termina una partida. Por otro, permite medir el efecto de las decisiones de diseño: una heurística admisible, una poda bien ubicada o una regla de ocupación pueden cambiar la ruta, el tiempo y el resultado. El documento busca que esas decisiones sean defendibles, no solo que el programa funcione.

Objetivos

- Modelar formalmente el tablero, el escenario, el estado, las acciones y la transición.
- Implementar y comparar BFS, DFS, UCS, A*, Beam Search, Minimax y poda alfa-beta.
- Separar el dominio puro de la interfaz y de la carga de archivos.
- Medir generados, expandidos, frontera, cortes, profundidad y tiempo.
- Verificar que los caminos devueltos sean válidos y que su costo sea coherente.
- Documentar resultados, complejidad y limitaciones con evidencia reproducible.

Alcance y no alcance

El alcance incluye una partida automática entre dos bandos, con ambos usando el mismo algoritmo seleccionado. No incluye aprendizaje automático, comunicación en red, niebla de guerra ni decisiones estocásticas. Tampoco se implementa una interfaz para elegir Minimax o alfa-beta: esos algoritmos existen y se ejecutan mediante la API de `Partida`, pero el selector visual solo expone `ALGORITMOS_BUSQUEDA`. Esta diferencia se documenta como limitación, no como una capacidad inexistente.

Contribuciones

- Un modelo de dominio inmutable, con validaciones explícitas y errores tempranos.
- Una capa de búsqueda genérica basada en `Problema`, `ProblemaTablero` y `ProblemaMultiobjetivo`.
- Cinco algoritmos de búsqueda y dos de decisión, con métricas homogéneas.
- Una partida automática que integra intercepción, transferencia, despeje de ruta y anti-oscilación.
- Una regla de ocupación que permite apilar aliados solo en la base propia.
- Una experimentación sobre cuatro escenarios y una discusión explícita de validez.

3. Arquitectura y flujo de una partida

El repositorio se organiza por responsabilidades. La regla de diseño es que `domain` no conozca pygame, archivos ni algoritmos; `search` y `decision` no conozcan la interfaz; y `app` sea el único punto que combine dominio, búsqueda y decisión. La Tabla 1 resume los paquetes y su función.

Paquete / archivo	Responsabilidad
domain/	Position, Terrain, Board, Unit, Scenario, GameState, acciones y reglas puras.
io/	Carga JSON, validación de campos y construcción del modelo de dominio.
search/	Problema, BFS, DFS, UCS, A*, Beam y heurísticas.
decision/	Utilidad, Minimax y poda alfa-beta.
metrics/	Métricas inmutables de búsqueda y decisión; contador mutable de decisión.
verification/	Verificación de caminos: inicio, meta, sucesores y costo.
app/	Partida automática y controlador entre vista y modelo.
ui/	Pygame: renderer, paneles, sprites y carga de assets.

Tabla 1. Organización modular del proyecto.

El flujo de una ejecución es: `main.py` carga un escenario con `load_scenario`, que valida el diccionario y devuelve el JSON; `construir_modelo` crea `Board`, `Scenario` y `GameState`; `Partida` recibe el escenario y el estado; en cada turno decide una acción, la aplica y actualiza el resultado; la UI consume `avanzar`, `posiciones_previas` y `resumen`. La Figura 1 resume el recorrido.

```
JSON → validate_scenario → Board + Scenario + GameState → Partida
→ decisiones → aplicar_accion → resultado_partida → UI
```

Figura 1. Flujo conceptual de carga, decisión y actualización.

La separación tiene una consecuencia práctica: los algoritmos de búsqueda reciben un `Problema`, no un `GameState`. Eso permite reutilizarlos para una unidad que va a su base, a la celda del recurso o a una celda adyacente a un portador enemigo. La decisión adversarial, en cambio, sí recibe el estado completo porque necesita anticipar movimientos, transferencias e intercepciones del rival.

4. Modelado formal del problema

Tablero como grafo ponderado

Sea un tablero de `F` filas y `C` columnas. El conjunto de celdas es `V = {0..F-1} × {0..C-1}`. Cada celda `v` tiene un terreno `T(v)`, una bandera `walkable(v)` y un costo de entrada `w(v) > 0` si es transitable. El grafo es `G = (V, E)`, con aristas entre celdas ortogonalmente adyacentes; `E` tiene como máximo `2FC` aristas. Un muro no pertenece a las aristas transitables. El costo de entrar a `v` es `w(v)`; la celda de origen no se cobra.

```
V = F × C E subconjunto de { (u,v) : |Δfila| + |Δcolumna| = 1 } w(v) > 0
```

Escenario

El escenario es la parte estática: tablero, bases, posición inicial del recurso, profundidad máxima de Minimax, límite de turnos y bloque opcional `prueba`. Se representa con `Scenario`, una dataclass frozen. El límite `max_turnos` vale 200 por defecto. `Scenario` valida que las bases y el recurso estén dentro del tablero y sobre terreno transitable, que existan las bases A y B y que la profundidad, si aparece, sea un entero positivo.

```
S = (tablero, bases, recurso, profundidad_maxima_minimax, max_turnos, prueba)
bases: {A -> Position, B -> Position} recurso en V max_turnos = 200
```

Estado

El estado dinámico es una tupla `G = (U, turno, portador, n, pases, bloqueo, bases)`. `U` es la tupla de unidades; cada unidad tiene `id`, `bando`, `tipo` y `posición`. `turno` indica a quién le corresponde jugar. `portador` es el id de la unidad que lleva el recurso, o `None`. `n` es el número de turno, que empieza en 1. `pases` cuenta pases consecutivos y `bloqueo` cuenta los turnos que faltan para permitir otra intercepción. `bases` se incluye para que la validación de ocupación y las transiciones puras no dependan de un objeto externo.

```
G = (U, turno, portador, n, pases, bloqueo, bases)
U = ( (id, bando, tipo, posición), ... ) turno en {A, B}
portador en ids(U) o {None} n ≥ 1 pases ≥ 0 bloqueo ≥ 0
```

Acciones y transición

El conjunto de acciones es `A = {Mover, Transferir, Interceptar, Pasar}`. `Mover(u, d)` desplaza una unidad a una celda ortogonal; `Transferir(origen, destino)` cambia el portador entre aliados adyacentes; `Interceptar(u)` arrebata el recurso a un portador enemigo adyacente; `Pasar()` cede el turno. La transición `T(s, a)` es determinista y devuelve un estado nuevo; no modifica el anterior. `aplicar_accion` comprueba primero que la acción pertenezca a `acciones_legales(s)` y luego construye el sucesor.

```
T : S × A → S        T(s,a) = s'        s' ≠ s en general
aplicar_accion(s,a) = s' si a en acciones_legales(s); si no, AccionInvalida
```

Terminalidad

La partida termina por victoria si el portador está en la base de su propio bando; por empate si `n > max_turnos` o si se acumulan cuatro pases consecutivos. La comprobación de victoria se hace antes que los empates. El resultado es un objeto `Resultado(terminada, ganador, motivo)`, con `ganador=None` en los empates.

```
victoria(s) <=> portador(s) ≠ None y posición(portador) = base(bando(portador))
empate_turnos(s) <=> n > 200        empate_pases(s) <=> pases ≥ 4
```

Espacio de estados

El estado completo es finito si se consideran el límite de turnos y los contadores, pero su tamaño crece de forma combinatorial: en el peor caso, hasta `N^U` distribuciones de unidades sobre `N` celdas, multiplicadas por el turno, el portador, los pases y el bloqueo. Por eso la búsqueda de rutas no explora el estado completo: resuelve el movimiento de una unidad y trata a las demás como obstáculos estáticos durante ese turno. Minimax y alfa-beta sí exploran el árbol de estados completos, limitados por la profundidad configurada.

5. Representación computacional del estado

El dominio usa dataclasses frozen. `Position` contiene `row` y `column` y rechaza valores negativos; `Terrain` contiene `name`, `walkable` y `cost`; `Board` contiene dimensiones, matriz de nombres de terreno y catálogo de terrenos; `Unit` contiene `id`, `bando`, `tipo` y `posicion`; `Scenario` y `GameState` se describieron arriba. La inmutabilidad evita que una búsqueda modifique el estado que está evaluando y hace que los sucesores sean valores independientes.

`GameState` valida tres invariantes: el turno pertenece a `{A,B}`; el portador, si existe, corresponde a una unidad real; y dos unidades solo pueden compartir celda si son del mismo bando y la celda es la base propia de ese bando. También calcula `contadores` por bando y congela `bases` y `contadores` con `MappingProxyType`. `Position` usa `row/column`; el JSON usa `fila/columna`; la traducción ocurre en `scenario_loader`.

La representación del estado de búsqueda es distinta de la representación del estado del juego. Para una ruta, basta `Position` y un conjunto de celdas bloqueadas: la unidad se mueve sola y las demás unidades son obstáculos durante ese turno. Para la decisión adversarial, se necesita `GameState` completo porque una acción puede mover una unidad, transferir el recurso o interceptarlo, y el turno cambia. Esta distinción es clave para explicar por qué la búsqueda es mucho más barata que Minimax.

Tipo	Campos principales	Invariante o uso
Position	row, column	No negativos; identifica celdas.
Terrain	name, walkable, cost	Transitable → costo positivo; muro sin costo.
Board	rows, columns, cells, terrain_types	Dimensiones, celdas y catálogo coherentes.
Unit	id, bando, tipo, posicion	Identidad y localización de una unidad.
Scenario	tablero, bases, recurso, profundidad, max_turnos, prueba	Configuración estática validada.
GameState	unidades, turno, portador, n, pases, bloqueo, bases	Estado dinámico inmutable y validado.
ResultadoBusqueda	camino, costo, exito, metricas	Resultado de una ruta y su evidencia.
ResultadoDecision	accion, valor, metricas	Acción elegida y valor Minimax.

Tabla 2. Tipos principales del modelo y su función.

6. Reglas del juego

Las reglas se implementan en `domain/rules.py`. La función `acciones_legales` calcula las acciones del bando en turno y `aplicar_accion` construye el sucesor. Ambas son funciones puras: no leen archivos ni dibujan. La Tabla 3 resume las acciones.

Acción	Precondición	Efecto	Costo / turno
Mover(u,d)	u del bando en turno; d ortogonal, transitable y no bloqueada.	u cambia de posición; recogida automática si d es el recurso y no hay portador.	Cuesta w(d) para la ruta; consume un turno.
Transferir(o,d)	o es el portador actual; d es aliado adyacente.	d pasa a ser portador; no hay movimiento.	Consume un turno; no cuesta terreno.
Interceptar(u)	u del bando en turno; portador enemigo adyacente; bloqueo = 0.	u pasa a ser portador; bloqueo se reinicia en 3.	Consume un turno; no cuesta terreno.
Pasar()	No hay otra acción legal.	Aumenta pases consecutivos; turno alterna.	Consume un turno; descuenta bloqueo.

Tabla 3. Acciones legales y efectos.

Ocupación y bases

Fuera de las bases, una celda ocupada por cualquier unidad bloquea el movimiento. La base A admite varias unidades A y la base B varias unidades B; un enemigo no puede apilarse allí. Una base enemiga vacía sigue siendo transitable para una sola unidad del otro bando. Esta regla se aplica en `GameState` (validación del estado inicial) y en `celdas_bloqueadas_para` (cálculo de rutas). Los aliados en la base propia no bloquean la búsqueda, la transferencia ni la ruta del portador. Un enemigo en la base sí bloquea.

La regla anterior es importante para `murosBandera`: `B1` está en la base B y `B2` lleva el recurso. Si los aliados en la base bloquearan, `B2` no podría entrar; como no bloquean, `B2` puede transferir a `B1` o entrar a la base. La validación inicial permite duplicados solo en la base propia y con el mismo bando; fuera de ella, o con bandos mezclados, lanza `ValueError`.

Recogida, transferencia e intercepción

La recogida es automática: si no hay portador y una unidad se mueve a la celda del recurso, esa unidad pasa a ser portadora. La transferencia solo se permite desde el portador actual hacia un aliado adyacente; el recurso no se mueve de celda, solo cambia de unidad. La intercepción solo se permite si el portador es enemigo, está adyacente y el bloqueo es cero. Tras interceptar, el bloqueo se fija en 3 y cualquier acción distinta de interceptar lo descuenta en uno. Una transferencia no cancela el bloqueo.

El bloqueo es global, no por unidad: mientras sea mayor que cero, ningún bando puede interceptar. Esto evita robos consecutivos y da tiempo a que el portador se aleje. En `robo.json` la secuencia observada es: `A1` intercepta (bloqueo 3), `B2` mueve (2), `A1` mueve (1), `B1` mueve (0) y `A1` entra a la base y gana. La traza coincide con la regla.

Pases y terminación

`Pasar` solo es legal cuando el bando no tiene ninguna otra acción. Cada pase incrementa `pases_consecutivos`; cualquier acción distinta lo reinicia a cero. Si se acumulan cuatro pases consecutivos, la partida termina en empate. El número de turno se incrementa en cada acción; si supera `max_turnos`, también hay empate. La victoria se comprueba antes que cualquiera de los dos empates.

Detalle de implementación
`resultado_partida` se evalúa al inicio y después de cada acción. Si el portador entra a su base, el resultado es victoria inmediata. Si se alcanza el límite de turnos o de pases, el resultado es empate. Una acción ilegal no produce un estado nuevo: `aplicar_accion` lanza `AccionInvalida`.

7. Costos y función de transición

El costo de una ruta es la suma de los costos de entrada de las celdas que se pisan, sin contar la celda inicial. La longitud es el número de aristas del camino. Por eso una ruta puede ser más corta en pasos y más cara en terreno, o al revés. La Tabla 4 muestra los terrenos definidos en los escenarios.

Terreno	Transitable	Costo de entrada	Observación
camino	Sí	1	Costo mínimo de los escenarios.
pasto	Sí	2	Costo intermedio.
bosque	Sí	4	Penaliza la ruta.
agua	Sí	5	Solo aparece en jsonPrueba.
pantano	Sí	7	Costo máximo transitable.
muro	No	—	No se puede entrar.

Tabla 4. Costos de terreno usados por el modelo.

```
costo(camino) = Σ w(v_i) para i = 1..L        longitud(camino) = L = |camino| - 1
      g(n) = costo real acumulado desde el inicio        f(n) = g(n) + h(n)
```

BFS y DFS no usan `w` para ordenar la exploración; BFS minimiza el número de aristas en un grafo no ponderado y DFS no garantiza optimalidad. UCS y A* sí usan `w`; A* además usa una heurística. Beam ordena por `h`, no por `g`, y puede perder la ruta de costo mínimo. En la partida automática, la elección entre unidades compara `resultado.costo`, mientras que la decisión de transferir compara `resultado.longitud` en turnos. Esta diferencia es deliberada: para decidir quién debe llevar el recurso importa el número de turnos, no solo el costo del terreno.

8. Algoritmos de búsqueda

Interfaz del problema

`Problema` es un protocolo con `inicio`, `es_objetivo(posicion)` y `sucesores(posicion)`. `ProblemaTablero` implementa el recorrido ortogonal: valida inicio y meta, rechaza obstáculos y celdas bloqueadas, y devuelve pares `(vecino, costo de entrada)`. `ProblemaMultiobjetivo` hereda el comportamiento y considera meta a cualquier celda de un conjunto no vacío. `objetivos_adyacentes` construye las celdas libres alrededor de una unidad enemiga.

Algoritmo	Estrategia	¿Óptimo en costo?	¿Completo?	Uso en TacticalGrid
BFS	Cola FIFO; expande por niveles.	No en grafo ponderado; sí en pasos.	Sí, grafo finito.	Ruta de una unidad.
DFS	Pila LIFO; profundiza.	No.	Sí, con visitados.	Ruta de una unidad.
UCS	Heap por g; costos positivos.	Sí.	Sí, con costos positivos.	Ruta de costo mínimo.
A*	Heap por f = g + h.	Sí, con h admisible y consistente.	Sí, con h admisible.	Ruta de costo mínimo.
Beam	Conserva k nodos por nivel según h.	No.	No garantizado.	Exploración acotada.

Tabla 5. Propiedades de los algoritmos de búsqueda.

BFS y DFS

BFS usa una cola `deque`, un conjunto de visitados y un diccionario de padres. Expande el nodo más antiguo; por eso encuentra la meta con el menor número de aristas. En un tablero con costos 1, 2, 4, 5 y 7, el menor número de pasos no implica el menor costo. DFS usa una pila y revierte los sucesores para respetar el orden determinista del problema. Puede encontrar una ruta válida, pero no tiene garantía de optimalidad y su ruta puede ser mucho más costosa.

UCS

UCS mantiene una cola de prioridad por costo acumulado `g`. Rechaza costos no positivos y descarta entradas obsoletas cuando el costo extraído es mayor que el mejor conocido. La primera vez que extrae la meta, el costo es óptimo para costos positivos. La implementación usa un contador de desempate para no comparar `Position` cuando los costos son iguales.

A*

A* ordena por `f(n) = g(n) + h(n)`. Con una heurística admisible y consistente, el primer nodo meta expandido tiene costo mínimo. La implementación usa `mejor_g`, `padre`, `cerrados` y un heap; descarta vecinos con `h = ∞`, lo que evita insertar ramas sin ruta. En `Partida`, la heurística es `crear_manhattan_multiple` escalada por el costo mínimo de terreno. El módulo también ofrece `crear_distancia_exacta`, pero la partida automática no la selecciona.

Beam Search

Beam avanza por niveles. En cada nivel expande el haz actual, genera candidatos, deduplica por posición conservando el mejor `g`, ordena por `h`, luego por `g` y finalmente por fila y columna, y conserva solo los `k` mejores. Las alternativas descartadas se pierden: no hay lista de respaldo. Por eso `k` reduce la exploración, pero la ruta puede no ser la de costo mínimo. En los escenarios probados, `k=1` fue el más barato en nodos, pero en `jsonPrueba` perdió una búsqueda y obtuvo un costo acumulado mayor.

Integración con la partida

En cada turno, `Partida._decidir_busqueda` hace lo siguiente: si hay una intercepción legal, la ejecuta; si el portador propio puede transferir a un aliado con una ruta claramente mejor, transfiere; si no, calcula la meta de cada unidad propia y busca una ruta. La meta es la base propia si la unidad es portadora; una celda adyacente al portador enemigo si el rival lleva el recurso; o la celda del recurso si nadie lo lleva. Un aliado del portador no tiene meta. Se elige la ruta de menor costo; si el primer paso devuelve a la unidad a su celda anterior, se replantea con el mismo algoritmo bloqueando esa celda. Si nada funciona, se intenta despejar la ruta del portador y, en último caso, se elige una acción legal de respaldo.

```
flujo de decisión de búsqueda:
 Interceptar legal → ejecutar
transferencia beneficiosa → Transferir
mejor ruta por unidad → Mover
sin ruta → despejar ruta del portador → movimiento sin retroceso → legales[0]
```

La comparación de transferencia usa turnos: `longitud_aliado + 1 < longitud_portador`; si el portador no tiene ruta, basta con que el aliado tenga una. Un aliado en la base gana de inmediato. La base bloqueada por un enemigo se trata como ruta infinita. Los aliados en la base propia no bloquean. Esta regla explica que en `murosBandera` la búsqueda termine con `B2 transfiere el recurso a B1`, mientras que Minimax puede terminar con `B2 -> base`; ambas acciones son legales y producen victoria de B.

9. Heurísticas

Una heurística `h(n)` estima el costo restante desde `n` hasta la meta. Para que A* sea óptimo, `h` debe ser admisible (no sobreestimar) y, en la implementación con cierre, consistente. Las heurísticas del proyecto se construyen una vez y se evalúan en tiempo constante, salvo la distancia exacta, que se precalcula.

Manhattan escalada

La distancia Manhattan entre `p` y la meta `g` es `|p.row - g.row| + |p.column - g.column|`. Como cada movimiento ortogonal cambia esa distancia en exactamente 1 y entrar a una celda cuesta al menos `c_min`, la función `h(p) = c_min * d_Manhattan(p,g)` nunca sobreestima. También es consistente: la diferencia de `h` entre dos vecinos es a lo sumo `c_min`, que es el costo mínimo de la arista. En los escenarios, `c_min = 1`, así que la heurística coincide con la Manhattan pura.

```
h(p) = c_min × ( |p.row - g.row| + |p.column - g.column| )
```

Manhattan multiobjetivo

Cuando la meta es un conjunto `G`, la heurística toma el mínimo de las distancias a cada objetivo: `h(p) = c_min * min_{g en G} d_Manhattan(p,g)`. El mínimo de funciones admisibles y consistentes sigue siendo admisible y consistente. Se usa para la base, el recurso y las celdas adyacentes al portador enemigo.

```
h(p) = c_min × min_{g en G} ( |p.row - g.row| + |p.column - g.column| )
```

Distancia exacta

`crear_distancia_exacta` ejecuta Dijkstra desde la meta sobre el grafo invertido y guarda el costo real mínimo de cada celda hasta la meta. La heurística resultante es perfecta: A* expande solo los nodos necesarios para la ruta óptima. El costo de construcción es `O(F*C*log(F*C))`; luego cada evaluación es `O(1)`. Esta heurística está implementada, pero `Partida` usa Manhattan multiobjetivo para A* y Beam. La distinción debe explicarse en la sustentación para no atribuir a la partida una heurística que no está seleccionada.

Heurística y optimalidad
Manhattan escalada es admisible y consistente con los costos del proyecto. La distancia exacta también lo es, pero es más costosa de construir. Beam no hereda la garantía de A*: aunque use una buena `h`, descartar nodos puede eliminar la única ruta óptima.

10. Decisión adversarial

Función de utilidad

La utilidad se evalúa desde la perspectiva de un bando MAX. Si el estado es terminal, el valor es exacto: `+1000` si gana MAX, `-1000` si pierde y `0` si hay empate. Si no es terminal, se combinan cuatro componentes: posesión, progreso del portador, control del recurso cuando nadie lo lleva y riesgo por adyacencia. El valor se recorta a `[-100, 100]` para que una victoria siempre domine a cualquier evaluación heurística.

```
U(s) = +1000 si gana MAX;  -1000 si gana MIN;  0 si empate
U(s) = clamp( U_posesión + U_progreso + U_control + U_riesgo, -100, 100 )
```

Si hay portador y es de MAX, se suma `20 + 30 * progreso`; si es de MIN, se resta. El progreso es `1 - d_Manhattan(portador, base) / ((filas-1)+(columnas-1))`. Si el portador es de MAX, las unidades de MIN adyacentes penalizan `5` cada una; si es de MIN, las unidades de MAX adyacentes premian `5` cada una. Este término de riesgo solo se aplica cuando el bloqueo de robo es cero, porque mientras está bloqueado ninguna intercepción es legal. Si no hay portador, se usa `15 * (cerca_MIN - cerca_MAX)`, donde `cerca` es la distancia normalizada mínima de las unidades del bando a la celda del recurso; si un bando no tiene unidades, su distancia se toma como 1.

```
progreso = 1 - d_Manhattan(portador, base) / ((F-1)+(C-1))
si portador de MAX:  +20 + 30*progreso  - 5*(enemigos adyacentes)
si portador de MIN:  -20 - 30*progreso  + 5*(aliados adyacentes)
sin portador:        15*(cerca_MIN - cerca_MAX)
```

La función usa Manhattan, no la distancia real en pasos. Por eso no modela muros, ocupación ni costos de terreno. Es una aproximación deliberada: permite evaluar rápido muchos estados, pero puede preferir una posición que geométricamente está cerca y estratégicamente está bloqueada. Esta es una limitación que debe explicarse al defender los resultados.

Minimax

Minimax recibe el estado, el escenario, la profundidad y el bando MAX. Si la raíz es terminal, devuelve acción `None` y la utilidad exacta. Si no, expande las acciones legales; cuando el turno coincide con MAX elige el máximo, y cuando no, el mínimo. La recursión se detiene al llegar a un terminal o a `restante = 0`, y evalúa con `evaluar_estado`. En empates se conserva la primera acción del orden determinista de `acciones_legales`.

```
minimax(s, d) = U(s)                                si terminal(s) o d = 0
  minimax(s, d) = max_a minimax(T(s,a), d-1)          si turno(s) = MAX
  minimax(s, d) = min_a minimax(T(s,a), d-1)          si turno(s) = MIN
```

Poda alfa-beta

Alfa-beta mantiene una ventana `[α, β]`: α es la mejor opción conocida para MAX y β la mejor para MIN. Si `α ≥ β`, la rama ya no puede mejorar la decisión del padre y se corta. La poda solo se contabiliza cuando quedan hermanos sin explorar. La implementación conserva el mismo criterio de desempate que Minimax, por lo que debe devolver la misma acción y el mismo valor con los mismos argumentos. La diferencia está en el número de nodos explorados.

```
alfabeta(s, d, α, β) = U(s)                                  si terminal(s) o d = 0
   MAX: v = max(v, alfabeta(...)); α = max(α, v); si α ≥ β, cortar
   MIN: v = min(v, alfabeta(...)); β = min(β, v); si α ≥ β, cortar
```

Profundidad, desempate y métricas

La profundidad efectiva sigue la precedencia: argumento explícito > `juego.profundidad_maxima_minimax` > 3. En los escenarios, `example` y `murosBandera` usan 4, `jsonPrueba` usa 5 y `robo` usa 3. Las métricas registran generados, expandidos, evaluados, cortes, hijos podados, profundidad alcanzada y tiempo. En la partida automática se llama a Minimax con el bando en turno como MAX; el rival actúa como MIN. Por eso cada bando maximiza su propia perspectiva y la utilidad es simétrica.

Minimax no es una búsqueda de ruta
Minimax explora acciones completas: mover, transferir, interceptar o pasar. No encuentra “el camino más corto” a la base; elige la acción que maximiza la utilidad dentro del horizonte. En cambio, BFS/DFS/UCS/A*/Beam resuelven una ruta para una unidad. Confundir ambas cosas es uno de los errores más frecuentes al explicar el proyecto.

11. Metodología experimental

Preguntas de evaluación

- ¿Los algoritmos de búsqueda encuentran una ruta válida y terminan la partida?
- ¿UCS y A* obtienen rutas de menor costo que BFS/DFS?
- ¿Cómo cambia Beam al variar k?
- ¿Minimax y alfa-beta eligen las mismas acciones y valores?
- ¿Alfa-Beta reduce nodos y tiempo sin cambiar la decisión?
- ¿La regla de transferencia y la ocupación de bases producen el comportamiento esperado?

Escenarios

La Tabla 6 resume los escenarios usados. Todos tienen `max_turnos = 200`. La columna “profundidad” es la profundidad de Minimax configurada en el JSON. `jsonPrueba` tiene 8×8 y tres unidades por bando; `robo` tiene 5×5 y tres por bando. No hay escenario de 20×20.

Escenario	Dimensiones	Unidades	Por bando	Portador inicial	Profundidad	Propósito
example	4×4	4	2	—	4	Caso base con terreno variado.
jsonPrueba	8×8	6	3	A2	5	Escala y ruta larga.
murosBandera	4×4	4	2	—	4	Muros, base y transferencia.
robo	5×5	6	3	B1	3	Intercepción y enfriamiento.

Tabla 6. Escenarios usados en la experimentación.

Procedimiento

Cada ejecución crea una `Partida` con el escenario y el algoritmo, y avanza hasta que `terminada` sea verdadera o hasta un límite de seguridad. Para los algoritmos de búsqueda se instrumenta `Partida._buscar` y se acumulan las métricas de todas las búsquedas internas: rutas de unidades, transferencias, despeje y anti-oscilación. Para Minimax y alfa-beta se leen las métricas de cada turno desde el historial. Las cifras de “gen” y “exp” de búsqueda son, por tanto, acumulados de varias búsquedas, no de una sola ruta. Las cifras de decisión son la suma de las decisiones turno a turno.

Los algoritmos son deterministas: no hay números aleatorios ni semillas. Los empates se resuelven conservando la primera acción del orden de `acciones_legales`. Cada camino exitoso se verifica con `verificar_camino`, que comprueba inicio, meta, sucesores válidos y costo coherente. Las partidas se ejecutaron en una sola máquina, con Python 3.12.6, sin contenedor ni paralelismo. Los tiempos son de pared y sirven como indicadores, no como una comparación estadística.

Métricas

Métrica	Búsqueda	Decisión	Interpretación
generados	Nodos insertados en la frontera.	Hijos generados en el árbol.	Trabajo de generación.
expandidos	Nodos extraídos y expandidos.	Nodos con acciones legales calculadas.	Trabajo de expansión.
frontera / evaluados	Mayor tamaño de la frontera.	Hojas evaluadas por la utilidad.	Memoria o costo de evaluación.
cortes / podados	—	Podas efectivas y hermanos no explorados.	Eficacia de alfa-beta.
profundidad	—	Nivel máximo visitado.	Alcance del horizonte.
tiempo	Segundos de pared.	Segundos de pared.	Costo observado.

Tabla 7. Métricas registradas por el proyecto.

Validación y amenazas a la validez

La validación funcional se hizo con ejecuciones repetibles y con comprobaciones directas: resultados esperados, traza de bloqueo de robo, validación de duplicados en bases, prueba headless de la UI y verificación de caminos. No existe una suite automatizada ni CI. Los resultados corresponden a una sola ejecución por combinación; los conteos son deterministas, pero los tiempos pueden variar. Los escenarios son pequeños comparados con un tablero 20×20, así que las conclusiones de rendimiento no se generalizan sin nuevas mediciones.

Otra amenaza es la instrumentación: la búsqueda acumula métricas de todas las llamadas internas de un turno, incluidas las que se hacen para comparar transferencias o replantear rutas. Por eso un algoritmo puede mostrar más generados que otro aunque su ruta principal sea similar. Para comparar costo de ruta se usa el costo acumulado de rutas exitosas; para comparar el resultado de la partida se usan ganador y turnos. La Tabla 6 y el Anexo A permiten reconstruir ambas lecturas.

12. Resultados

Resultado global

Escenario	Algoritmos de búsqueda	Decisión adversarial	Acción final relevante
example	B gana en 6 turnos (8/8)	B gana en 6 (Minimax, Alfa-Beta)	B1 entra a la base.
jsonPrueba	A gana en 18; DFS en 24 (8/8)	No ejecutado	A2 entra a la base.
murosBandera	B gana en 10 (8/8)	B gana en 10 (Minimax, Alfa-Beta)	Búsqueda: B2 transfiere a B1. Decisión: B2 entra a la base.
robo	A gana en 5 (8/8)	A gana en 5 (Minimax, Alfa-Beta)	A1 entra a la base tras interceptar.

Tabla 8. Resultados globales por escenario y familia de algoritmo.

La lectura principal es que todos los algoritmos terminan las partidas y coinciden en el ganador. Las diferencias aparecen en los turnos, en el costo de las rutas y en el trabajo realizado. `jsonPrueba` es el caso más claro: BFS, UCS, A* y todas las variantes de Beam ganan en 18 turnos, pero DFS necesita 24. `murosBandera` es el caso donde la regla de transferencia cambia la acción final: la búsqueda transfiere a `B1`, que ya está en la base; Minimax y alfa-beta mueven a `B2` a la base, que también es legal porque los aliados pueden apilarse allí.

Rendimiento de búsqueda

La Tabla 9 selecciona las combinaciones más informativas para comparar costo y exploración. El Anexo A contiene las 32 combinaciones de búsqueda y las 6 de decisión. Los valores de “gen”, “exp” y “ms” son acumulados de las búsquedas internas de la partida; el costo es la suma de los costos de las rutas exitosas. No deben interpretarse como una única ruta.

Escenario	Algoritmo	G	T	Gen	Exp	Frontera	ms	Costo	Long.
example	BFS	B	6	88	53	4	7.68	65	22
example	DFS	B	6	111	87	6	5.10	69	28
example	UCS	B	6	80	53	4	2.28	59	24
example	A*	B	6	63	58	4	1.96	59	24
jsonPrueba	BFS	A	18	1313	1019	12	19.15	383	190
jsonPrueba	DFS	A	24	1306	757	22	15.55	1398	597
jsonPrueba	UCS	A	18	1192	878	16	18.11	348	191
jsonPrueba	A*	A	18	770	526	16	11.79	348	191
jsonPrueba	Beam k=1	A	18	508	215	4	5.89	427	211
jsonPrueba	Beam k=8	A	18	1700	957	11	23.77	392	209
murosBandera	BFS	B	10	83	67	3	1.10	29	23
murosBandera	A*	B	10	58	66	3	1.17	29	23
robo	BFS	A	5	80	51	5	0.95	22	21
robo	DFS	A	5	79	49	5	0.89	24	23
robo	A*	A	5	46	35	5	0.90	22	21

Tabla 9. Selección de métricas de búsqueda. ms = tiempo de búsqueda acumulado.

Rendimiento de decisión

Minimax y alfa-beta eligieron el mismo ganador y los mismos turnos en los tres escenarios donde se ejecutaron. Alfa-Beta redujo de forma consistente los nodos generados, los nodos evaluados y el tiempo. La Tabla 10 muestra los acumulados por partida. En `example`, alfa-beta pasó de 3452 a 732 generados y de 2742 a 458 evaluados; en `robo`, de 2948 a 607 generados y de 2599 a 475 evaluados. Las podas efectivas confirman que la ventana α-β se cerró en muchas ramas.

Escenario	Algoritmo	G	T	Gen	Eval	Podas	Prof.	ms	Valor
example	Minimax	B	6	3452	2742	0	4	314.99	1000
example	Alfa-Beta	B	6	732	458	562	4	83.67	1000
murosBandera	Minimax	B	10	557	349	0	4	47.55	1000
murosBandera	Alfa-Beta	B	10	395	225	68	4	39.01	1000
robo	Minimax	A	5	2948	2599	0	3	409.39	1000
robo	Alfa-Beta	A	5	607	475	532	3	91.92	1000

Tabla 10. Métricas de decisión acumuladas por partida.

Discusión por escenario

example: pasos frente a costo

Todos los algoritmos ganan con B en 6 turnos. BFS y Beam usan 22 movimientos acumulados y costo 65; DFS usa 28 y 69; UCS y A* usan 24 y 59. La diferencia entre 22 pasos y 24 pasos no implica que BFS

sea mejor en costo: UCS y A* aceptan dos pasos adicionales para pagar menos terreno. Este es el ejemplo más limpio de por qué `longitud` y `costo` son métricas distintas.

jsonPrueba: DFS y el costo de no optimizar

Los ocho algoritmos de búsqueda ganan con A; siete lo hacen en 18 turnos y DFS en 24. El costo acumulado de DFS es 1398, frente a 348 de UCS y A*. BFS obtiene 383 y Beam k=1 427. El caso muestra que DFS puede ser rápido en tiempo de pared y, a la vez, producir rutas mucho más caras. También muestra que Beam k=1 explora menos (508 generados) pero pierde una búsqueda y obtiene un costo mayor; no es un algoritmo óptimo.

murosBandera: transferencia y base

La partida dura 10 turnos con todos los algoritmos. En la búsqueda, `B2` recoge el recurso y lo transfiere a `B1`, que está en la base; la transferencia es legal porque son aliados adyacentes y la base propia admite varios aliados. En Minimax y alfa-beta, `B2` entra a la base con el recurso y gana; también es legal. La comparación con la versión anterior, donde B necesitaba 14 turnos, no es un resultado actual: fue una consecuencia del diseño previo de transferencia. El valor 10 es el que debe defenderse con el código actual.

robo: intercepción y enfriamiento

La traza de `robo` confirma la regla: `A1` intercepta en el primer turno, el bloqueo queda en 3; `B2` mueve y baja a 2; `A1` mueve y baja a 1; `B1` mueve y baja a 0; `A1` entra a la base y gana. El bloqueo no lo cancela una transferencia ni un pase: solo una nueva intercepción lo reinicia. Este escenario es la evidencia funcional del contador `turnos_robo_bloqueado`.

Respuesta a las preguntas

- Los algoritmos de búsqueda encuentran rutas válidas y terminan las partidas en los cuatro escenarios.
- UCS y A* obtienen el menor costo en los casos donde el terreno importa; DFS puede duplicar o cuadruplicar el costo.
- Beam reduce la exploración, pero k pequeño puede perder la ruta óptima; k=1 fue el caso más evidente.
- Minimax y alfa-beta coinciden en acción y valor; alfa-beta reduce nodos y tiempo.
- La transferencia y la ocupación de bases explican el resultado de murosBandera y la traza de robo.

13. Análisis de complejidad

Sea `N = F*C` el número de celdas, `U` el número de unidades, `b` el factor de ramificación y `d` la profundidad. En el tablero ortogonal, una celda tiene a lo sumo cuatro vecinos, así que `b ≤ 4` para la búsqueda de una unidad. En decisión, `b` no se acota con `4U`: además de los movimientos, las transferencias y las intercepciones agregan acciones legales. Las reglas y la utilidad se ejecutan por nodo y también cuestan.

Componente	Tiempo	Espacio	Comentario
BFS	O(N + E) = O(N)	O(N)	Cola, visitados y padres.
DFS	O(N + E) = O(N)	O(N)	Pila, visitados y padres.
UCS	O((N+E) log(N+E))	O(N+E)	Heap con entradas repetidas.
A*	O(b^d) peor; O(E log V) con heap	O(N)	Óptimo con h admisible y consistente.
Beam	O(d·k·b·log(k·b))	O(k·b)	No completo ni óptimo.
Minimax	O(b^d)	O(d·b)	Explora el árbol completo hasta d.
Alfa-Beta	O(b^d) peor; O(b^(d/2)) ideal	O(d·b)	Depende del orden de acciones.

Componente	Tiempo	Espacio	Comentario
acciones_legales / aplicar_accion	O(U²)	O(U)	Recorren unidades y vecinos.
evaluar_estado / resultado_partida	O(U)	O(1) adicional	Recorren unidades y bases.
verificar_camino	O(L)	O(1) adicional	L = longitud del camino.

Tabla 11. Complejidad de los componentes principales.

El espacio de estados completo merece una advertencia. Si se ignoran los contadores, hay hasta `N^U` formas de ubicar `U` unidades en `N` celdas, y a eso se suman turno, portador, pases y bloqueo. Para 20×20 y varias unidades, el árbol de Minimax crece muy rápido. La profundidad limitada y la poda alfa-beta son necesarias; aun así, `jsonPrueba` con profundidad 5 no se completó en la experimentación de decisión. En cambio, la búsqueda de una unidad trabaja sobre `N` posiciones y es tratable incluso en tableros mayores.

La complejidad de `Partida` combina ambas capas. El docstring de `_decidir_busqueda` la resume como `O(U*S + A*S + M*S)`, donde `S` es el costo del algoritmo de búsqueda, `A` el número de aliados adyacentes y `M` el número de movimientos legales. En la práctica, cada turno puede ejecutar varias búsquedas: una por unidad, una por transferencia y una de replanteo. Esa es la razón por la que las métricas del Anexo A se presentan como acumulados.

14. Limitaciones y trabajo futuro

Limitaciones actuales

- **UI incompleta para decisión:** `ALGORITMOS_DISPONIBLES = ALGORITMOS_BUSQUEDA`; Minimax y Alfa-Beta no aparecen en el selector, aunque `Partida` los acepta y fueron probados por API.
- **Bloque `prueba` no ejecutado:** el JSON conserva `prueba`, pero la función que lo resolvía se eliminó al reescribir el controlador. No se usa para la partida automática.
- **Sin suite de pruebas:** no hay test runner, linter ni CI. La verificación se hizo con scripts temporales y comprobaciones directas.
- **Cobertura de escenarios:** no hay un escenario de 20×20. El mayor es 8×8; los resultados de rendimiento no se generalizan.
- **Sin dibujo de ruta ni exportación:** el renderer define un color de camino, pero no dibuja la ruta; la UI no exporta métricas.
- **README vacío:** la documentación de uso no está en el repositorio; este PDF la sustituye parcialmente.
- **Utilidad aproximada:** usa Manhattan, no distancia real; no modela muros, ocupación ni costos.
- **Decisión limitada:** Minimax no usa tabla de transposición, ordenamiento de acciones ni profundización iterativa; el costo crece rápido.
- **Resultados de una sola ejecución:** los conteos son deterministas, pero los tiempos dependen del hardware y de Python.
- **Heurística exacta no seleccionada:** `crear_distancia_exacta` existe, pero la partida usa Manhattan multiobjetivo.
- **Trazabilidad de IA:** los docstrings registran intervención de IA y algunas validaciones aparecen pendientes; este documento no sustituye esa revisión.

Trabajo futuro

- Exponer Minimax y Alfa-Beta en la UI y permitir comparar sus métricas en vivo.
- Agregar escenarios 20×20 con tres o más unidades por bando y obstáculos más densos.
- Implementar una suite de pruebas automatizadas para reglas, búsqueda, decisión y validación.
- Dibujar la ruta calculada y exportar métricas a CSV o JSON desde la interfaz.
- Permitir elegir la heurística de A* (Manhattan o distancia exacta) y comparar su efecto.

- Agregar ordenamiento de acciones y tabla de transposición para alfa-beta.
- Repetir los experimentos varias veces y reportar medias, medianas y dispersión.
- Ejecutar el bloque `prueba` del escenario para pruebas de regresión guiadas por JSON.
- Revisar y cerrar las validaciones estudiantiles registradas en los docstrings.

15. Conclusiones

- TacticalGrid modela un juego determinista, por turnos y con información perfecta; la separación entre `Scenario`, `GameState` y acciones permite razonar sobre el dominio sin depender de pygame.
- Los cinco algoritmos de búsqueda resuelven los escenarios, pero no son equivalentes: UCS y A* minimizan costo; BFS minimiza pasos; DFS no garantiza optimalidad; Beam cambia exploración por riesgo de perder la mejor ruta.
- La comparación entre longitud y costo es esencial: en `jsonPrueba`, DFS usa 24 turnos y costo 1398, mientras UCS y A* usan 18 turnos y costo 348.
- Minimax y Alfa-Beta coinciden en ganador, turnos, acción y valor; Alfa-Beta reduce de forma clara nodos y tiempo, lo que valida la poda.
- La regla de apilamiento aliado en la base y la transferencia explican el resultado de `murosBandera` en 10 turnos; el valor 14 pertenece a una versión anterior y no debe usarse como resultado actual.
- Las limitaciones están acotadas: la UI no expone decisión, no hay 20×20 ni suite automatizada, y la utilidad usa Manhattan. Son líneas claras de trabajo futuro, no fallas ocultas.

16. Referencias

Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.

Pearl, J. (1984). Heuristics: Intelligent Search Strategies for Computer Problem Solving. Addison-Wesley.

Korf, R. E. (1990). Real-Time Heuristic Search. Artificial Intelligence, 42(2–3), 189–211.

Python Software Foundation. (2025). The Python Language Reference (3.12). https://docs.python.org/3/

Pygame Community Edition. (2025). pygame-ce Documentation. https://pyga.me/

fpdf2. (2025). fpdf2 Documentation. https://py-pdf.github.io/fpdf2/

Anexo A. Métricas crudas

Las tablas de este anexo contienen todas las combinaciones ejecutadas. Para búsqueda, `gen`, `exp`, `frontera` y `ms` son acumulados de todas las búsquedas internas de la partida; `costo` es la suma de los costos de las rutas exitosas y `long` la suma de sus longitudes. Para decisión, los valores son la suma de las decisiones turno a turno. La columna `G` indica el ganador y `T` el número de turnos ejecutados.

Lectura correcta de las métricas
No se debe comparar el costo acumulado de una partida de búsqueda con el valor de utilidad de Minimax: son escalas distintas. El primero suma costos de rutas de terreno; el segundo es el valor Minimax de la raíz, en el rango [-1000, 1000].

A.1 Búsqueda

Escenario	Algoritmo	G	T	Gen	Exp	Frontera	ms	Costo	Long
example	BFS	B	6	88	53	4	7.68	65	22
example	DFS	B	6	111	87	6	5.10	69	28
example	UCS	B	6	80	53	4	2.28	59	24
example	A*	B	6	63	58	4	1.96	59	24
example	Beam k=1	B	6	53	27	3	1.02	65	22
example	Beam k=2	B	6	62	36	4	1.07	65	22
example	Beam k=4	B	6	63	37	4	1.02	65	22
example	Beam k=8	B	6	63	37	4	1.41	65	22
jsonPrueba	BFS	A	18	1313	1019	12	19.15	383	190
jsonPrueba	DFS	A	24	1306	757	22	15.55	1398	597
jsonPrueba	UCS	A	18	1192	878	16	18.11	348	191
jsonPrueba	A*	A	18	770	526	16	11.79	348	191
jsonPrueba	Beam k=1	A	18	508	215	4	5.89	427	211
jsonPrueba	Beam k=2	A	18	820	369	6	10.08	393	209
jsonPrueba	Beam k=4	A	18	1300	646	9	16.54	392	209
jsonPrueba	Beam k=8	A	18	1700	957	11	23.77	392	209
murosBandera	BFS	B	10	83	67	3	1.10	29	23
murosBandera	DFS	B	10	83	70	3	1.36	29	23
murosBandera	UCS	B	10	82	66	3	1.33	29	23
murosBandera	A*	B	10	58	66	3	1.17	29	23
murosBandera	Beam k=1	B	10	54	51	3	1.20	26	20
murosBandera	Beam k=2	B	10	59	60	3	1.19	29	23
murosBandera	Beam k=4	B	10	59	61	3	1.59	29	23
murosBandera	Beam k=8	B	10	59	61	3	1.23	29	23
robo	BFS	A	5	80	51	5	0.95	22	21
robo	DFS	A	5	79	49	5	0.89	24	23
robo	UCS	A	5	80	50	6	1.28	22	21
robo	A*	A	5	46	35	5	0.90	22	21
robo	Beam k=1	A	5	45	21	4	0.68	22	21
robo	Beam k=2	A	5	62	31	4	0.84	22	21
robo	Beam k=4	A	5	74	43	5	1.33	22	21
robo	Beam k=8	A	5	74	43	5	1.21	22	21

Tabla A.1. Métricas acumuladas de las 32 combinaciones de búsqueda.

A.2 Decisión adversarial

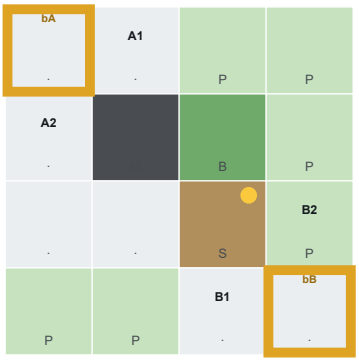
Escenario	Algoritmo	G	T	Gen	Eval	Podas	Prof	ms	Valor
example	Minimax	B	6	3452	2742	0	4	314.00	1000
example	Alfa-Beta	B	6	732	458	562	4	82.00	1000
murosBandera	Minimax	B	10	557	349	0	4	47.00	1000
murosBandera	Alfa-Beta	B	10	395	225	68	4	39.00	1000
robo	Minimax	A	5	2948	2599	0	3	409.00	1000
robo	Alfa-Beta	A	5	607	475	532	3	91.00	1000

Tabla A.2. Métricas acumuladas de Minimax y alfa-beta.

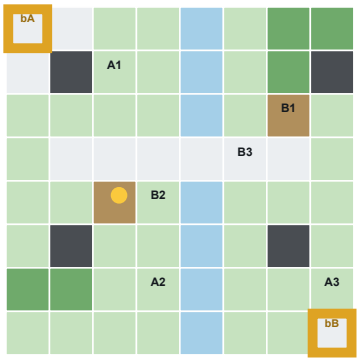
Anexo B. Diagramas de los escenarios

Los diagramas muestran la posición inicial de cada escenario. El borde dorado marca las bases; el punto amarillo, el recurso; las etiquetas A1/B1 identifican unidades. Los números de fila y columna siguen el orden del JSON: fila 0 es la primera fila y columna 0 la primera columna. Las celdas se colorean según el terreno.

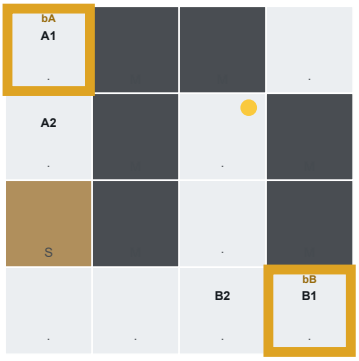
example.json · 4×4



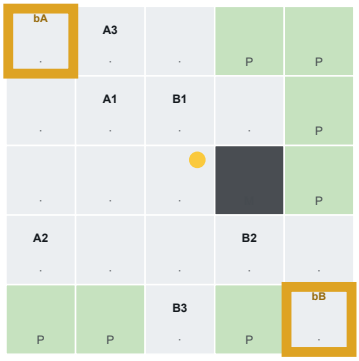
jsonPrueba.json · 8×8



murosBandera.json · 4×4



robo.json · 5×5



Convenciones

camino (1) · pasto (2) · bosque (4) · pantano (7) · agua (5) · muro (no transitable)

bA / bB = base; punto amarillo = recurso; A1/B1 = unidades. Una base puede contener varios aliados del mismo bando; un enemigo no puede apilarse allí.

Figura B.1. Estados iniciales de example, jsonPrueba, murosBandera y robo.

Anexo C. Contrato del escenario JSON

El validador exige `version = "1.0"` y las claves `version`, `mapa`, `tipos_terreno`, `terreno`, `bases`, `recurso`, `unidades`, `turno` y `juego`. `prueba` es opcional. Las posiciones se expresan con `fila` y `columna`; el dominio las convierte a `row` y `column`. Las posiciones de bases, recurso y unidades deben estar dentro del mapa y sobre terreno transitable.

Clave	Tipo / contenido	Regla principal
version	texto	Debe ser exactamente "1.0".
mapa	filas, columnas	Enteros positivos; la matriz debe coincidir.
tipos_terreno	nombre → {transitable, costo}	Transitable exige costo positivo; muro sin costo.
terreno	matriz de nombres	Cada nombre debe existir en tipos_terreno.
bases	A y B → {fila, columna}	Dentro del mapa y transitables.
recurso	{fila, columna}	Dentro del mapa y transitable.
unidades	lista de {id, bando, tipo, fila, columna}	Ids únicos; bando A/B; ocupación válida.
turno	"A" o "B"	Bando que juega primero.
juego.portador_recurso	id o null	Si no es null, debe existir.
juego.profundidad_maxima_minimax	entero positivo opcional	Si aparece, se usa como profundidad.
prueba	objeto opcional	Se conserva, pero no se ejecuta en la versión actual.

Tabla C.1. Claves y validaciones del contrato JSON.

La regla de ocupación se valida dos veces: en `validate_scenario` y en `GameState`. Varias unidades pueden compartir celda solo si es la base propia del bando de todas ellas. Fuera de esa base, o con bandos mezclados, se rechaza. El archivo `ejemploerror.json` valida correctamente a pesar de su nombre: su celda de agua es transitable; el nombre no implica invalidez.

Anexo D. Pseudocódigo

El pseudocódigo resume la lógica, no reproduce el código fuente. Las versiones completas están en `tacticalgrid/search/` y `tacticalgrid/decision/`.

D.1 BFS

```
BFS(problema):
  cola ← [inicio]; visitados ← {inicio}; padre ← {inicio: None}
  mientras cola no vacía:
    n ← cola.extraer_primero()
    si problema.es_objetivo(n): devolver reconstruir(padre, n)
    para (v, costo) en problema.sucesores(n):
      si v no en visitados:
        visitados.add(v); padre[v] ← n; cola.añadir(v)
  devolver fallo
```

D.2 A*

```
A*(problema, h):
  frontera ← [(h(inicio), 0, inicio)]; g[inicio] ← 0
  mientras frontera no vacía:
    n ← extraer_min_por_f(frontera)
    si n es meta: devolver reconstruir(padre, n)
    para (v, costo) en problema.sucesores(n):
      nuevo ← g[n] + costo
      si nuevo < g[v]:
        g[v] ← nuevo; padre[v] ← n
        insertar(frontera, (nuevo + h(v), v))
  devolver fallo
```

D.3 Beam Search

```
Beam(problema, h, k):
  haz ← [inicio]
  mientras haz no vacío:
    candidatos ← {}
    para n en haz:
      si n es meta: devolver ruta
      para (v, costo) en problema.sucesores(n):
        si v no expandido: candidatos[v] ← mejor g y h
    haz ← los k candidatos con menor h (desempate: g, fila, columna)
  devolver fallo
```

D.4 Minimax

```
Minimax(s, d, MAX):
  si terminal(s) o d = 0: devolver U(s)
  si turno(s) = MAX:
    devolver max_a Minimax(T(s,a), d-1, MAX)
  si no:
    devolver min_a Minimax(T(s,a), d-1, MAX)
```

D.5 Alfa-Beta

```
AlfaBeta(s, d, α, β, MAX):
  si terminal(s) o d = 0: devolver U(s)
  si turno(s) = MAX:
    v ← -∞
    para a en acciones(s):
      v ← max(v, AlfaBeta(T(s,a), d-1, α, β, MAX))
      α ← max(α, v); si α ≥ β: romper
    devolver v
  si no:
    v ← +∞
    para a en acciones(s):
      v ← min(v, AlfaBeta(T(s,a), d-1, α, β, MAX))
      β ← min(β, v); si α ≥ β: romper
    devolver v
```

Anexo E. Trazabilidad, validación y uso de IA

E.1 Ejecución

El punto de entrada es `python main.py` desde la raíz del repositorio; carga `scenarios/example.json` mediante una ruta relativa a `main.py`. La dependencia declarada es `pygame-ce>=2.5.8`. Para pruebas sin ventana se usó el controlador de video `dummy` de SDL. Los experimentos de este documento se ejecutaron con `PYTHONPATH=.` y sin escribir archivos de caché en el repositorio.

```
python main.py
python -m compileall tacticalgrid
PYTHONPATH=. python <script_temporal_de_medicion>.py
SDL_VIDEODRIVER=dummy SDL_AUDIODRIVER=dummy python main.py
```

E.2 Evidencia funcional

- Los cuatro escenarios terminan con un ganador y un motivo `victoria`.
- Cada camino exitoso pasa por `verificar_camino`: inicio, meta, sucesores y costo.
- La traza de `robo` confirma el bloqueo 3→2→1→0 y la intercepción de A1.
- La traza de `murosBandera` confirma la transferencia de B2 a B1 en la base.
- La validación de duplicados acepta aliados en la base propia y rechaza duplicados fuera de ella o con bandos mezclados.
- La UI headless se probó con dos unidades A en la base A y dos B en la base B sin errores.

E.3 Uso de IA

El código incluye en sus docstrings los campos `AI usage`, `AI intervention` y `Student validation`. La mayoría de las funciones declara asistencia de IA y algunas validaciones aparecen como pendientes. Este documento se apoya en la ejecución reproducible y en la verificación de caminos; no afirma que esas validaciones pendientes ya estén cerradas. Para la sustentación se recomienda revisar esos campos y actualizar el estado de validación que corresponda.

E.4 Reproducibilidad

Los conteos de nodos, ganadores y turnos son deterministas para los escenarios y algoritmos probados. Los tiempos son de pared y pueden variar entre equipos. Para reproducir una fila del Anexo A basta cargar el escenario indicado, crear `Partida` con el algoritmo y avanzar hasta `terminada`. La interpretación de las métricas acumuladas está en la sección 11.